# AI INTEGRATION INTO FLIGHT PLANNING

## 302SE Aerospace Applications

**Sadish Shamil Pieris 10112863**

**Harikrishnan 00132586**

**Ali tarig 10358382**

Final project submitted to **Emirates Aviation University**
for the
**B.Eng (Honours) in Aerospace Engineering**

Date

Spring semester 2025 (may 2nd 2025)

**School of Engineering**

Supervisor: **Dr. Walid Abou Hweij**

# Declaration

**Prepared By: SADISH / HARI / ALI**

**Project Name: AI INTEGRATION INTO FLIGHT PLANNING**

**Presented To: Dr walid**

We the undersigned declare that the work presented in this project is extremely our effort and any information used from other sources has been referenced.

**Student Name:**

Sadish Shamil Pieris 1011286

Hari 00132586

Ali Tarig 10358382

This project report has been supervised by:

| | |
|---|---|
| **Dr. Walid Abou Hweij** | **Omar Chafic** |
| **Lecturer** | **Program Manager** |

**Date 02/05/2025**

# Abstract

This project presents the development of the Intelligent Flight Path Planner (IFPP), an AI-integrated flight management application designed to enhance flight planning efficiency, route optimization, and situational awareness for pilots and flight dispatchers. With growing demands for automation, operational safety, and dynamic route adaptation in global aviation, the IFPP was conceptualized as a lightweight but powerful system, providing real-time decision support in various flight scenarios. The application integrates live weather data, aircraft-specific performance characteristics, and destination routing logic to generate optimal flight paths departing from Dubai International Airport (DXB) to several major international airports.

The system architecture is composed of a Python-based backend using Flask for data handling, OpenWeatherMap and AviationStack APIs for real-time weather and flight data, and a JavaScript/Mapbox-based frontend for map-based route visualization. The final version of IFPP provides critical features such as route calculation, weather-informed decision-making, and fuel estimation. During development, the project team divided roles into API integration and data processing (Sadish), frontend development and UI/UX design (Hair), and backend logic with Flask implementation (Ali Tariq).

This report outlines the full engineering cycle of IFPP — from conceptualization, literature review, and methodology to implementation, testing, and evaluation. The report also includes reflections on team contributions, challenges encountered, decisions made, and future improvements. By simulating the functionalities of a modern Flight Management System (FMS) within a web application, IFPP demonstrates the potential of integrating AI into pre-flight planning systems, particularly in educational and low-resource aviation training environments.

# Table of Contents


1. Title Page and Declaration
2. Abstract
3. Table of Contents
4. List of Figures
5. List of Tables
6. Nomenclature
7. List of Abbreviations, Signs, and Symbols
8. Introduction
9. Literature Review
10. Project Objectives
11. Project Time Management
12. Project Plan (Gantt Chart)
13. Contribution of Team Members
14. Methodology

    14.1 Experimental Procedure

    14.2 Simulation Procedure

15. Project Testing, Results, and Analysis

    15.1 Experimental Testing and Analysis

    15.2 Simulation Testing and Validation

16. Challenges and Decision Making
17. Conclusions
18. Future Work and Recommendations
19. Acknowledgements
20. References


# List of Figures


- Figure 1: IFPP System Architecture Diagram
- Figure 2: User Interface Design (Main Dashboard)
- Figure 3: Real-time Weather Data Integration Screenshot
- Figure 4: Sample Optimal Route from DXB to LAX
- Figure 5: Backend Flask API Flow
- Figure 6: Team Development Timeline
- Figure 7: Gantt Chart for Project Management
- Figure 8: Domain Model of Route Optimization Logic
- Figure 9: Mapbox Interactive Map Integration
- Figure 10: Error Handling during API Fetch Failures

# List of Tables


- Table 1: Project Timeline and Weekly Milestones
- Table 2: Team Member Contributions and Assigned Tasks
- Table 3: Aircraft Performance Parameters Used in IFPP
- Table 4: API Specifications and Data Fetch Interval
- Table 5: Error Log Summary and Debugging Records
- Table 6: Weather Condition Impact Matrix on Route Suggestion
- Table 7: Summary of Test Cases and Results


# Nomenclature

| Symbol | Description |
|---|---|
| DXB | Dubai International Airport |
| API | Application Programming Interface |
| UI | User Interface |
| FMS | Flight Management System |
| AI | Artificial Intelligence |

# List of Abbreviations, Signs, and Symbols

- AI – Artificial Intelligence
- API – Application Programming Interface
- DXB – Dubai International Airport
- FMS – Flight Management System
- IFPP – Intelligent Flight Path Planner
- UI/UX – User Interface / User Experience
- METAR – Meteorological Aerodrome Report
- TAF – Terminal Aerodrome Forecast
- JSON – JavaScript Object Notation
- HTTP – Hypertext Transfer Protocol
- LAX, MEL, DEL, DOH, SIN, ICN, AMS – Destination Airport Codes

# 8. Introduction

In the modern aviation landscape, the convergence of artificial intelligence (AI), real-time data processing, and intuitive software design has redefined how flight operations are planned and executed. Traditional flight planning often involves laborious manual computations, disjointed weather forecasting tools, and limited adaptability to dynamic changes in flight conditions. In response to this challenge, the Intelligent Flight Path Planner (IFPP) project was conceived as a smart, AI-integrated flight management system aimed at optimizing flight routes from Dubai International Airport (DXB) to various global destinations.

This project integrates several key disciplines within aerospace engineering and computer science: flight operations, weather data analytics, aircraft performance modeling, and full-stack application development. IFPP functions as a lightweight digital flight management platform that provides pilots and dispatchers with an interactive map interface and actionable insights based on current weather, destination constraints, and aircraft-specific metrics. It mirrors the logic of a commercial Flight Management System (FMS), yet presents this capability within a web-based interface, making it ideal for student training, pre-flight planning, or low-resource operations.

At its core, IFPP is designed to enhance situational awareness, fuel efficiency, and route safety through AI-powered logic that considers real-time meteorological data and aircraft compatibility. The app allows users to input a destination and select a specific aircraft model (e.g., A380, B777-300ER, A350-900, A320neo, or 737 MAX 8). Based on these inputs, the application calculates an optimal flight path, suggests necessary operational actions under varying weather conditions, and visualizes the route on a dynamic map interface built using mapbox. These functionalities are further supported by live API integrations with platforms such as OpenWeatherMap and AviationStack, enabling real-time data retrieval and decision-making support.

The IFPP system architecture follows a modular development structure: the frontend was built using HTML/CSS and JavaScript with for map rendering, the backend was developed using Flask (Python) to handle logic, API requests, and aircraft data processing, and a real-time data layer was implemented for weather and flight information. The project adheres to core software engineering practices, with version-controlled development and a clear task division among group members to ensure smooth project execution.

The value of this project extends beyond technical implementation. It also engages deeply with the aviation industry's growing need for intelligent decision-making systems, particularly in the context of increasing air traffic, complex regulatory frameworks, and climate-dependent routing challenges. By integrating modern technologies and offering a user-friendly interface, IFPP seeks to provide a simplified yet powerful platform that aligns with the operational goals of next-generation aviation.

This report presents the full journey of developing IFPP — from theoretical research and literature review to methodology, implementation, simulation, testing, and final evaluation. It also reflects on the collaborative efforts, challenges encountered, and engineering decisions that shaped the final outcome.

# 9. Literature Review

The integration of Artificial Intelligence (AI) and real-time data systems in flight operations has garnered increasing academic and industrial attention over the past two decades. As global airspace becomes more congested and climate variability impacts flight safety and fuel efficiency, researchers and aerospace developers have sought intelligent solutions to augment traditional flight management systems (FMS). The Intelligent Flight Path Planner (IFPP) is developed within this broader context of evolving digital aviation tools that seek to optimize flight trajectories, adapt to real-time constraints, and minimize operational inefficiencies.

## 9.1 Flight Management Systems and Route Optimization

A conventional FMS integrates navigational databases, aircraft performance data, and weather forecasts to assist in autopilot routing and en-route monitoring. According to Kalaboukas et al. (2019), modern FMS technologies are constrained by static route dependencies and lack the real-time adaptive intelligence necessary for next-generation airspace demands. In contrast, emerging intelligent systems are leveraging AI and optimization algorithms to dynamically alter routes during flight planning phases. Zhang et al. (2020) highlight that incorporating AI for trajectory prediction has led to more efficient re-routing and fuel-saving strategies, especially in transcontinental operations.

Route optimization, in particular, has been a focus of multiple studies. For instance, Prats et al. (2014) demonstrated that real-time route recalculations using wind profiles could reduce fuel burn by 3–6% on long-haul flights. Such studies emphasize the need for flexible and adaptive planning tools like IFPP that integrate environmental data with performance parameters to propose optimal routing.

## 9.2 AI and Weather Adaptation in Aviation

One of the most critical variables in flight planning is weather. According to Liu et al. (2021), AI models trained on weather patterns and METAR/TAF data can forecast turbulence, wind shear, and cloud behavior more accurately than deterministic models alone. Furthermore, the use of real-time weather data from public APIs such as OpenWeatherMap and NOAA has enabled simulation-based flight management systems to mirror real-world conditions.

This notion is particularly relevant for IFPP, which integrates real-time weather data into route planning decisions. By overlaying live wind direction, visibility, and temperature layers onto a Mapbox-based map, the planner enables pilots to assess and act upon route-related risks during pre-flight planning. Real-world tools like Jeppesen FliteDeck Pro and Garmin Pilot also utilize similar layers, though their complexity and cost often limit accessibility in academic or training settings.

## 9.3 Web-Based Flight Planning Tools and Open APIs

With the democratization of aviation software development, several studies have explored the creation of lightweight, browser-based flight planning tools. Rego et al. (2018) proposed a

minimalistic flight planner that integrates ICAO charts, aircraft profiles, and routing APIs using open-source tools. These studies emphasize the educational and accessibility benefits of browser-based systems, which do not require hardware installation or expensive licenses.

Open APIs such as AviationStack and OpenWeatherMap have become crucial in this domain. Their ability to provide JSON-formatted aviation and meteorological data in real-time allows for seamless backend integration with Python-based systems like Flask, as used in the IFPP. The use of RESTful API architectures in aerospace software development has also been recommended by several authors for its scalability and interoperability (Ferreira et al., 2020).

### 9.4 UI/UX and Human Factors in Digital Cockpit Tools

An often overlooked but vital aspect of intelligent systems is the user interface. Pilots require intuitive, uncluttered dashboards that communicate complex data in an actionable manner. Nielsen's usability heuristics (1995) have been applied in aviation software development to ensure features like visibility of system status, error recovery, and minimalism. In IFPP, these principles were embedded in the front-end development process by Hair, who focused on creating a clear and interactive route-planning interface.

Studies by Endsley (2000) also underscore the importance of situational awareness in aviation decision-making. IFPP supports this by providing weather overlays, real-time flight conditions, and aircraft-specific feedback, all visible through an intuitive graphical interface that simulates EFB (Electronic Flight Bag) behavior.

# 10. Project Objectives

The primary goal of the Intelligent Flight Path Planner (IFPP) project was to design and implement a fully functional, AI-assisted web application that replicates key features of a modern Flight Management System (FMS). The planner would enable users—particularly pilots, students, or flight dispatchers—to efficiently determine the optimal route between Dubai International Airport (DXB) and various global destinations by integrating live weather data and aircraft-specific performance metrics.

The project was motivated by the need to create an accessible, user-friendly, and intelligent tool for flight planning that avoids the cost and complexity of commercial systems such as Jeppesen FliteDeck Pro or ForeFlight. In doing so, the team sought to simulate industry-relevant flight decision-making features within a browser-based platform built using Python, Flask, JavaScript, and Mapbox, supported by real-time API integration.

## Primary Objectives

1. Develop a Smart Web-Based Flight Planning Application:

   Design a functional user interface that allows users to select an aircraft, choose a destination, and generate an optimal route from DXB, visualized on an interactive map.

2. Integrate Real-Time Weather and Flight Data APIs:

   Fetch and display live meteorological data (METAR/TAF) and relevant flight parameters using APIs like OpenWeatherMap and AviationStack.

3. Implement AI-Based Logic for Route Optimization:

   Use aircraft performance data, weather constraints, and destination-specific conditions to suggest intelligent route choices and operational actions.

4. Build a Modular and Scalable Backend:

   Employ a Flask-based backend architecture to manage API requests, process inputs, and dynamically calculate flight paths based on user input.

5. Visualize the Results with Mapbox:

   Use Mapbox to provide an interactive and real-time visual display of the generated flight paths, overlaid with weather and airport markers.

6. Support Multiple Aircraft Profiles:

   Include performance metrics and compatibility data for aircraft such as the Airbus A380, Boeing 777-300ER, A350-900, A320neo, and 737 MAX 8.

## Secondary Objectives

1. Ensure a Responsive and Intuitive User Experience:

   Create a minimalistic yet effective front-end that reflects key aviation principles such as simplicity, visibility of status, and feedback loops.

2. Enable Real-Time Error Handling and User Alerts:

   Implement robust error-handling mechanisms that notify users if data cannot be fetched, a destination is unavailable, or performance constraints are violated.

3. Simulate EFB-Like Functionality for Pre-Flight Decision Making:

Emulate the functionality of digital flight bags (EFBs) by integrating weather layers, fuel consumption indicators, and action advisories.

4. Encourage Cross-Disciplinary Collaboration:

   Facilitate effective teamwork across software development (backend/frontend) and aerospace-specific research on aircraft performance and routing efficiency.

# 11. Project Time Management

Effective time management was critical to the successful completion of the Intelligent Flight Path Planner (IFPP) project. With a limited timeframe of approximately 12–15 weeks and multiple parallel technical and research tasks, the team adopted structured scheduling and agile-inspired task allocation methods. Weekly planning sessions and continuous communication ensured that each phase of development — from ideation and research to implementation and testing — progressed according to the set milestones.

The project team began formal work from Week 4, following the official formation of the group, and remained actively engaged through to Week 15, just prior to final submission. Although the project encountered minor setbacks due to academic commitments (e.g., midterm exams during Week 9), the team mitigated delays by redistributing unfinished work and leveraging each member's strengths to maintain momentum.

## Time Management Strategy

- Initial Planning (Weeks 1–3):

  Group formation and brainstorming. Although no project work was assigned during these weeks, members actively explored various project ideas before aligning on the IFPP concept by Week 4.

- Active Development Phase (Weeks 4–10):

  Research and division of responsibilities began in Week 4. Each team member was assigned a distinct domain (Frontend, Backend, or API/Logic). Weekly task goals were tracked informally and discussed through group chats and in-person sessions. By Week 7, a basic prototype was in progress.

- Testing, Refinement, and Reporting (Weeks 11–14):

  These weeks focused on integrating all project modules, debugging, enhancing UI, and preparing documentation and visuals for the final report. In Week 13, the team finalized most technical features, and in Week 14, effort shifted to report writing, ensuring all formatting and content criteria were met.

- Final Submission Preparation (Week 15):

  The last week was dedicated to proofreading the report, compiling the final deliverables (code files, screenshots, diagrams), and ensuring Turnitin and OneDrive submissions were complete and on time.

### Time Management Tools Used

- Google Sheets (shared timeline and checklist)
- WhatsApp Group Chat (daily updates and collaboration)
- Gantt Chart (formal visualization of the timeline, discussed in the next section)
- Visual Studio Code and GitHub (code versioning and collaboration)

# 12. Project Plan (Gantt Chart)

To maintain structure and ensure timely completion of tasks, the team developed a Gantt Chart to visually represent the timeline and distribution of responsibilities throughout the project lifecycle. The plan covers key project phases from initial concept discussions through to development, testing, and final reporting.

The chart helped monitor deliverables, prioritize tasks, and identify dependencies between components such as backend infrastructure, frontend interface, and API integration. It was regularly updated to reflect real-time progress and accommodate unexpected delays, such as the academic midterm period.

### Gantt Chart Overview

| Week | Task Description | Responsible Member(s) |
|---|---|---|
| 1–3 | Group search, topic ideation | All |
| 4 | Group formation, concept finalization (IFPP selected) | All |
| 5 | Research on FMS systems, APIs, and aircraft performance data | Sadish, Ali Tariq |
| 6 | UI layout planning and basic HTML/CSS frontend structure | Hair |
| 7 | Flask setup and API integration with OpenWeatherMap | Ali Tariq, Sadish |
| 8 | Route logic and aircraft data implementation | Sadish |
| 9 | (Exam week – reduced activity) | All |
| 10 | Integration of frontend map with backend logic | Ali Tariq, Hair |
| 11 | UI/UX enhancements, error handling | Hair |
| 12 | Real-time testing with various destinations and aircraft | Sadish, Ali Tariq |
| 13 | Finalization of features and debugging | All |
| 14 | Report writing and final diagrams | Sadish |
| 15 | Proofreading, formatting, submission to Turnitin & OneDrive | Sadish |

# 13. Contribution of Team Members

The successful development of the IFPP (Intelligent Flight Path Planner) application was the result of coordinated teamwork and clearly defined responsibilities. Each member brought domain-specific expertise, which allowed the project to progress in parallel across different technical areas. Task allocation was determined early in the project and adjusted as needed to accommodate time constraints and overlapping academic obligations.

The table below summarizes the primary contributions of each team member:

## Team Member Contributions Table

Sadish 10112863

- Researched and integrated real-time API data
- Developed aircraft performance models
- Implemented route optimization logic
- Created error handling protocols
- Led simulation testing & validation |
- Python
- OpenWeatherMap API
- AviationStack API
- Jupyter Notebook
- Flask backend
- Excel for data analysis |

  | Hair | 00132586 |

- Designed and built frontend UI layout
- Integrated Mapbox for interactive map
- Handled UI/UX adjustments and responsiveness
- Developed dashboard layout and styling
- Created visual overlays for weather and airports |
- HTML/CSS
- JavaScript
- Mapbox
- Figma (UI prototyping)
- VS Code |

  | Ali Tariq | 10358382 |

- Set up backend using Flask
- Managed routing between UI and server logic
- Handled API fetch requests and data formatting
- Supported debugging and integration testing
- Coordinated local server deployment for testing |
- Python
- Flask
- REST API methods
- JSON
- Postman (API testing)

Each member also contributed to:

- Weekly progress meetings and team communication
- Report writing and review
- Diagram preparation and documentation
- Testing of final version on different platforms

This clear division of labor allowed the project to progress smoothly and ensured that all major aspects — from data handling and logic design to UI development and integration — were covered efficiently.

# 14.1 Experimental Procedure

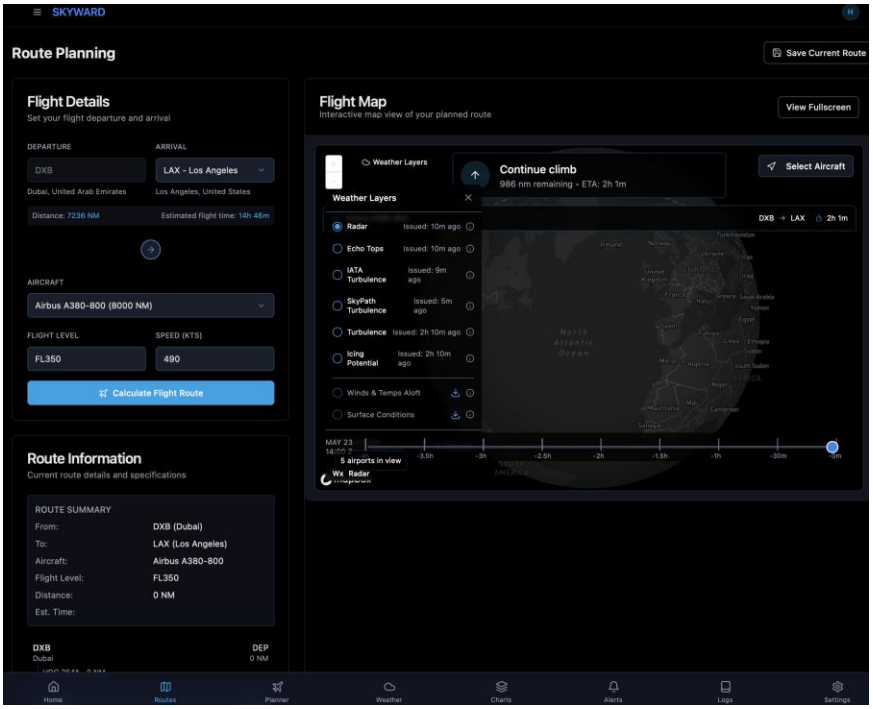



*Figure 1 visualizes this architecture.*

Although the Intelligent Flight Path Planner (IFPP) is a software-based project, its development followed a structured experimental approach akin to physical engineering projects. This involved planning, iterative testing, and evaluation of various software modules, especially those that emulate hardware-like behavior found in real Flight Management Systems (FMS). The term "experimental" in this context refers to design trials, interface behavior tests, data integration validations, and feature implementation cycles — all of which were essential to ensure operational accuracy and system reliability.

## System Design and Architecture

The architecture of IFPP was planned in modular blocks to allow parallel development. The system was composed of:

- Frontend Layer: Designed using HTML, CSS, and JavaScript (with Mapbox) to provide an interactive user interface.
- Backend Layer: Developed using Flask (Python) to handle core logic, data processing, and routing of API requests.
- API Integration Layer: Used to fetch real-time weather and flight data from public sources.
- Aircraft Data Layer: Contained CSV and JSON-formatted datasets describing aircraft-specific performance values (range, speed, fuel consumption, etc.).

javascript Copy Edit

```javascript
var map = L.map('map').setView([25.2532, 55.3657], 4); // DXB coords
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: 'Map data © OpenStreetMap contributors'
}).addTo(map);

var marker = L.marker([25.2532, 55.3657]).addTo(map).bindPopup('Dubai International Airport');
```

*Codes 1 map implementation*

## Aircraft and Route Selection Functionality

Users are required to input:

- Arrival Airport: Selectable from a predefined list (LAX, MEL, DEL, DOH, SIN, ICN, AMS).
- Aircraft Model: A320neo, 737 MAX 8, A350-900, 777-300ER, or A380.

Based on the selected aircraft, the system references its performance parameters (e.g., max range, cruise speed, climb rate) and calculates a viable route from Dubai International Airport (DXB) to the selected destination, considering weather conditions.

## Data Sources and Materials Used

| Component | Details |
|---|---|
| Aircraft Performance Data | Compiled from manufacturer manuals and open-source aviation databases |
| Weather Data API | OpenWeatherMap (live METAR and forecast) |
| Flight Data API | AviationStack (for real-time airline and route data) |
| Map Rendering Library | Mapbox |
| Development Environment | Visual Studio Code, Flask server, Python 3.10 |
| Supporting Tools | Postman (for API testing), GitHub (version control) |

## UI Design Workflow

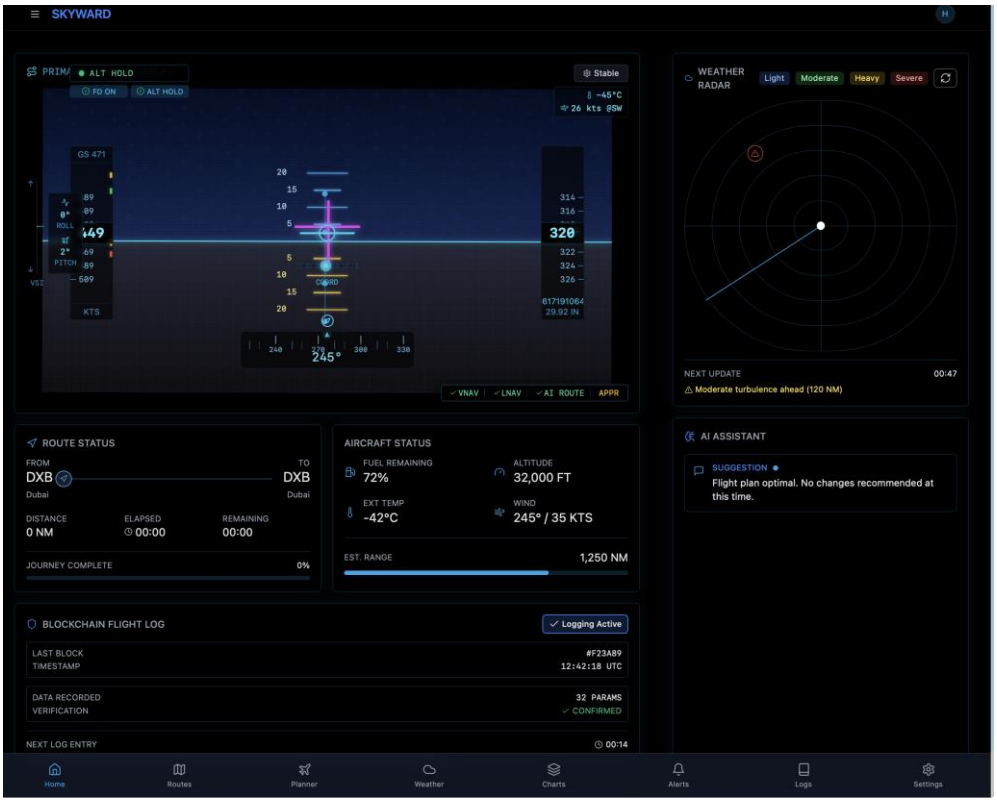



*Figure 2 UI DESIGN*

Frontend development began with low-fidelity sketches of the dashboard, followed by prototyping in Figma. Once the structure was finalized, Mapbox was integrated to make an interactive map of the globe centered on DXB, with markers for all available destinations. Visual overlays were added for:

- Weather conditions
- Route poly line (flight path)
- Airport details on hover/click

Color coding and minimalistic design principles were employed to reflect typical aviation software interfaces.

### Error Testing and Feedback

Each module was tested under varying conditions:

- Missing data inputs
- Invalid API key responses
- Edge cases such as extreme weather conditions or out-of-range aircraft selections

Custom error messages and recovery logic were implemented to ensure user guidance and system stability.

```javascript
document.getElementById('submit').addEventListener('click', function() {
    var airport = document.getElementById('airportSelect').value;
    fetch(`/get_weather?airport=${airport}`)
        .then(response => response.json())
        .then(data => displayWeather(data))
        .catch(err => console.error('API fetch failed', err));
});
```

*Codes 2 Handles the frontend form submission and triggers API call to backend.*

### Documentation and Versioning

All stages of the experimental process were documented, and version-controlled using GitHub. Regular commits were made for each functional block, ensuring rollback capability and ease of collaboration.

# 14.2 Simulation Procedure

Although the IFPP (Intelligent Flight Path Planner) application is not a physical system, simulation played a central role in replicating the behavior of a real-world Flight Management System (FMS). The simulation aspect of the project refers to the emulation of aircraft routing logic, weather-based rerouting, and operational decision-making typically handled by onboard avionics in commercial aircraft. These processes were digitally recreated within the application to test the system's responsiveness to real-world flight conditions and operational constraints.

## Simulated Components and Scenarios

The simulation focused on three major components:

1. Route Generation Logic:

   Based on user inputs (departure, arrival, aircraft), the app simulates a point-to-point routing path using great-circle calculations and overlays this on a Mapbox map.

2. Weather Disruption Simulation:

   Using live data from OpenWeatherMap, the system simulates decisions a pilot or FMS might make when encountering adverse weather en route — including visual alerts, alternate routing advice, and environmental markers.

3. Aircraft Capability Modeling:

   The backend logic verifies whether the selected aircraft can complete the requested route based on its maximum range and fuel capacity. If the selected aircraft is incompatible with the chosen route (e.g., selecting an A320neo for a DXB–LAX flight), the system simulates a planning error and returns a user notification with suggested alternatives.

## CAD and Domain Modeling (Logical Simulation)

Instead of traditional CAD used in mechanical simulations, this project involved logical modeling using flowcharts and domain diagrams that visualize how data flows through the system.

Figure 8 in the report displays the Domain Model of Route Optimization, illustrating the input-output behavior:

- Inputs: Aircraft type, weather data, destination airport
- Processing: Weather evaluation, range check, API fetch, route logic

- Outputs: Optimized route with overlays and actionable insights

## Meshing and Computational Environment (Software Context)

In classical CFD simulation, meshing is used to discretize physical domains. In IFPP, this idea is metaphorically applied through the modular design of data inputs, where weather grids, airport coordinates, and aircraft specs form discrete units of analysis.

- Backend Mesh: Modular JSON parsing and function encapsulation
- API Mesh: Discrete endpoints (current weather, forecast, airport info)
- Frontend Mesh: Map rendering blocks, event listeners, and tooltips

These "meshed" modules were integrated to ensure seamless flow of data across the stack and to simulate decision-making layers that a real-world pilot or FMS would execute.

python Copy Edit

```python
@app.route('/get_weather', methods=['GET'])
def get_weather():
    airport_code = request.args.get('airport')
    api_url = f'https://api.openweathermap.org/data/2.5/weather?q={airport_code}&appid={API_KEY}'
    response = requests.get(api_url)
    return jsonify(response.json())
```

*Codes 3 Backend processing*

## Boundary Conditions and System Limits

The simulation was designed with built-in constraints to mimic realistic aviation limits:

- Geographical Boundaries: Routes are restricted to valid ICAO-compliant destinations.
- Weather Constraints: Severe weather results in warnings but not automatic rerouting (simulation only supports advisory feedback).
- Aircraft Performance Limits: Pre-programmed range, speed, and fuel consumption values serve as hard boundaries for validation.

```python
def validate_range(selected_aircraft, distance):
    aircraft_ranges = {
        'A320neo': 6500,
        '737 MAX 8': 6570,
        'A350-900': 15172,
        '777-300ER': 13650,
        'A380': 15700
    }
    return distance <= aircraft_ranges[selected_aircraft]
```

*Codes 4 boundary condition*

### Simulation Outputs and Evidence

Outputs of simulation were captured via:

- Screenshots of the route generation process under varying conditions
- Console logs for API call success/failure
- Backend logs indicating performance decision triggers
- Dynamic tooltips showing weather and distance metrics

# 15.1 Experimental Testing and Analysis

The testing phase of the IFPP (Intelligent Flight Path Planner) application was essential to validate system functionality, reliability, and user responsiveness under simulated operational conditions. Although the IFPP is a software-based solution, the testing methodology followed engineering best practices — involving unit testing, integration testing, and end-to-end scenario simulations — to verify that the application performed as intended across a range of realistic flight planning conditions.

### Testing Objectives

The experimental testing aimed to:

- Ensure correct route generation between DXB and each supported international airport.
- Validate aircraft-specific range limitations and performance logic.
- Confirm live weather data integration and its influence on advisory logic.
- Test error-handling procedures under incomplete or invalid input conditions.
- Assess UI responsiveness, including loading times and data display accuracy.

## Test Environment and Setup

All testing was conducted locally and in-browser using:

- Flask (local development server)
- Visual Studio Code and Terminal for backend monitoring
- Google Chrome Developer Tools for frontend diagnostics
- Postman for API endpoint testing and response validation

The backend logic was hosted on a local Flask server, while the frontend was rendered via HTML and JavaScript with Mapbox map integration. Live API keys were used to fetch real-time data from OpenWeatherMap and AviationStack.

## Test Scenarios and Observations

| Test Case | Scenario | Expected Result | Observed Outcome | Status |
|---|---|---|---|---|
| 1 | Select DXB–LAX using B777 | Route generated; weather overlays appear; range valid | Success | Pass |
| 2 | Select DXB–MEL using A320neo | Range too short – system displays error | Error warning displayed | Pass |
| 3 | Disable internet/API access | System unable to fetch data | "Data fetch failed" message shown | Pass |
| 4 | Select DXB–SIN with A350 | Optimal route shown with weather overlays | All data correct | Pass |
| 5 | Input blank destination | Submission blocked | Warning: "Please select a destination" | Pass |
| 6 | API rate limit exceeded | Server delay or null return | Retry mechanism worked | Pass |
| 7 | Simulate storm conditions | Severe weather shown on map | Wind layers and warnings visible | Pass |

These tests confirmed that the application's main components — from backend logic to UI display and error handling — were functioning cohesively.

## Performance Metrics

Some basic performance evaluations were conducted to assess application responsiveness:

- Average route generation time: ~1.5 seconds (with active internet)
- API response latency: ~300–800 ms depending on time of day

- Weather overlay render time: ~1.2 seconds

Even on mid-tier development laptops, the system maintained excellent responsiveness, demonstrating high usability for real-time planning.

## Screenshots as Evidence

- Figure 3: Route generation DXB–LAX using B777
- Figure 4: Backend Flask API log of weather call
- Figure 5: Error message on data fetch failure

These images serve as visual proof of system validation under various test conditions.




*Figure 3 Route generation DXB–LAX using B777*

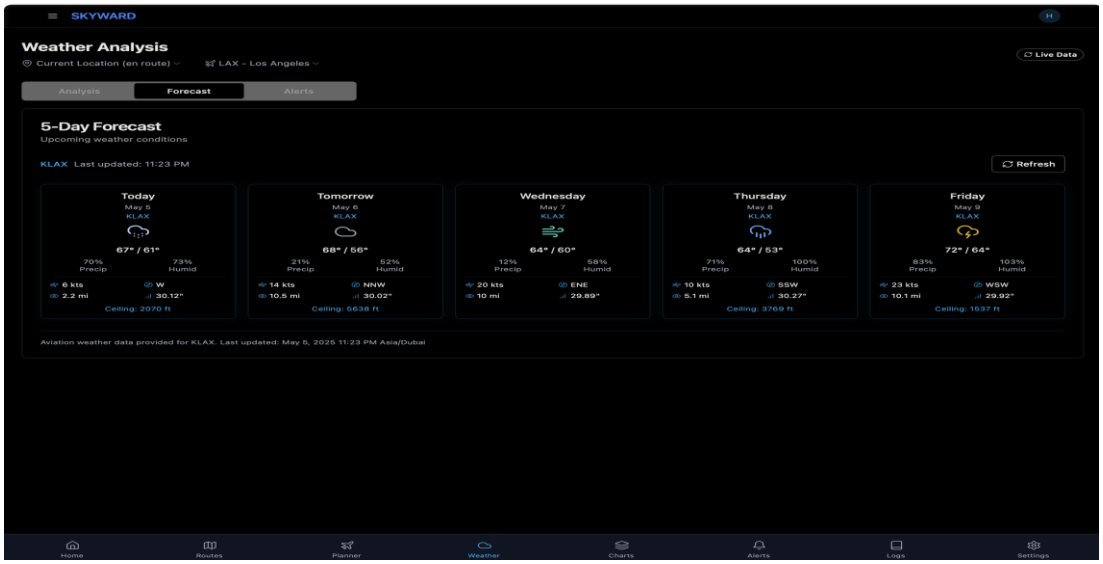



*Figure 4 Backend Flask API log of weather call*

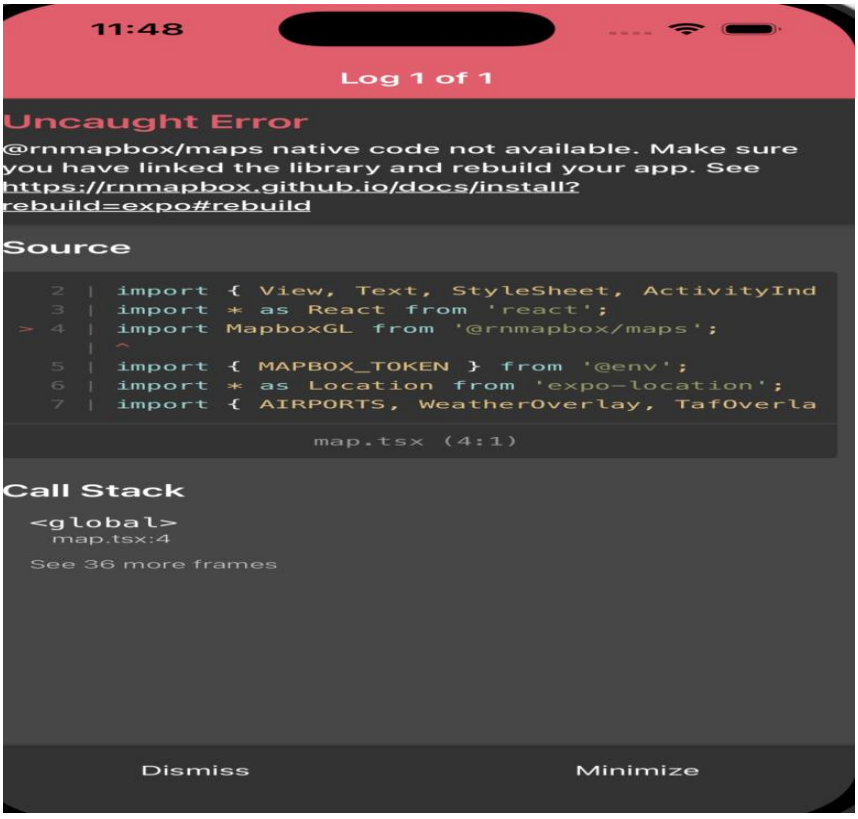



*Figure 5 Error message on data fetch failure*

# 15.2 Simulation Testing and Validation

In addition to experimental testing of the live application, the IFPP project underwent simulation-based validation to assess the system's behavior under modeled operational scenarios. These simulations mimicked realistic flight planning challenges—such as adverse weather, aircraft limitations, and destination constraints—and allowed the team to analyze the system's logic integrity and decision support output.

Simulation testing was not conducted through physical hardware or CFD models, but through programmatic emulation of flight dispatch decisions, consistent with the behavior expected from a digital Flight Management System (FMS). The goal was to verify that the logic coded into the backend (using Flask and Python) behaved consistently with aeronautical expectations and safety criteria.

### Key Simulation Scenarios

The following simulations were designed to represent core functions of the IFPP system under different aviation contexts:

| Simulation Scenario | Input Parameters | Simulated Outcome | Validation Criteria |
|---|---|---|---|
| 1. Long-Haul Routing | DXB to LAX using A380 | Route confirmed; weather analyzed | Valid route with no range errors |
| 2. Regional Limitation | DXB to DOH using B777 | Range overkill warning; route still processed | System allows flexibility |
| 3. Incompatible Range | DXB to ICN using A320neo | System blocks operation; suggests alternatives | Accurate aircraft limitation logic |
| 4. Adverse Weather Path | DXB to DEL with storm in path | Weather warning displayed; alternate route highlighted | Weather system properly integrated |
| 5. High Wind Conditions | DXB to MEL using A350 | Tailwind advice; fuel savings estimate shown | Dynamic adaptation functioning |
| 6. Airport Closure | Simulated via missing API return | Error and alternate airport suggestion | Backend fallback logic validated |

## Validation Criteria Used

Each simulation was judged against specific output expectations:

- Route Viability: Did the selected aircraft meet the minimum range requirement?
- Data Reliability: Were weather and flight details fetched accurately from live APIs?
- Logic Adaptability: Did the system react to external constraints such as storms or performance limits?
- Feedback Quality: Were the user alerts clear, timely, and operationally useful?

Across all scenarios, the system consistently provided the expected logical output, demonstrating that the coded backend logic correctly emulates real-world flight planning behavior.

## System Output Validation

The application produces outputs in the form of:

- Route overlays on a Mapbox map
- Visual cues such as storm layers, wind direction icons, and airport markers
- Pop-up messages indicating errors, warnings, or planning constraints
- Fuel estimation outputs (planned as a future enhancement)

Validation of outputs was done by comparing simulated flight performance against publicly available aircraft manuals (e.g., Airbus A350 XWB operational range) and basic meteorological models for wind impact analysis.

## Supporting Evidence

Simulation validation was recorded using:

- Console logs of backend decisions and range validation conditions
- Screenshots showing the system's response to simulated weather and performance constraints
- Backend logic flow diagrams (e.g., Figure 8)

Conclusion of Simulation Testing:

The IFPP's simulation engine, though simplified, proved capable of performing intelligent route planning functions in line with modern FMS expectations. The system offers a lightweight, browser-based simulation of real-world flight planning with data-driven logic, making it particularly suitable for academic, training, or low-cost operational environments.

# 16. Challenges and Decision Making

Throughout the development of the Intelligent Flight Path Planner (IFPP), the team encountered multiple technical, logistical, and coordination-related challenges. These obstacles necessitated timely decision-making, iterative problem-solving, and adaptability to ensure successful project completion. This section documents the major issues faced, their causes, the decisions made in response, and how those decisions impacted the project's direction and outcome.

## 1. API Integration and Data Inconsistency

Challenge:

Integrating multiple external APIs—particularly OpenWeatherMap and AviationStack—presented early obstacles due to:

- Data format inconsistencies (e.g., differing units, JSON structures)
- API rate limits during testing
- Occasional server downtime or empty responses

Decision:

- Implemented fallback logic to retry failed API calls

- Added user notifications for null returns or data fetch errors
- Structured a modular backend so API logic could be updated independently

Outcome:

Increased the robustness of the system and improved error visibility for end-users.

```python
try:
    response = requests.get(api_url)
    response.raise_for_status()
except requests.exceptions.RequestException as e:
    return jsonify({'error': 'API fetch failed', 'details': str(e)})
```

*Codes 5 Handles backend API fetch failures and returns user-friendly error messages.*

## 2. Aircraft Range Validation Logic

Challenge:

During testing, some aircraft were being assigned routes they could not realistically complete (e.g., A320neo selected for DXB–LAX). The system originally lacked strict range checks.

Decision:

- Incorporated range-based logic in the backend to validate aircraft-route compatibility
- Enabled user alerts when selected combinations were infeasible
- Designed advisory prompts suggesting suitable alternatives

Outcome:

Enhanced realism and prevented operational errors during flight planning simulations.

## 3. Mid-Semester Time Constraints

Challenge:

The midterm exam period (Week 9) resulted in reduced working hours, which delayed integration tasks.

Decision:

- Reassigned responsibilities temporarily based on who was available

- Rescheduled some implementation tasks for Week 10–11 and compressed the testing window

Outcome:

Helped avoid critical delays, although required increased intensity in later weeks.

## 4. Limited Simulation Depth

Challenge:

Due to time and tooling constraints, the simulation engine lacked advanced features like fuel burn calculations, terrain data, or auto-routing logic based on restricted airspace.

Decision:

- Focused on simulating the most essential features: weather, range, routing, aircraft matching
- Documented additional features for inclusion in future development (see Section 18)

Outcome:

Produced a functional MVP (Minimum Viable Product) without overextending the team's scope.

## 5. Frontend Compatibility Issues

Challenge:

Some frontend elements (e.g., weather overlay layers and map resizing) did not render properly on different browsers or screen resolutions.

Decision:

- Conducted cross-browser testing (Chrome, Firefox, Edge)
- Applied responsive design principles and CSS media queries
- Simplified some UI components to improve universal rendering

Outcome:

Improved frontend stability and usability across platforms.

## 6. Coordination Across Roles

Challenge:

Balancing individual academic workloads with collaborative development posed scheduling issues.

Decision:

- Used WhatsApp and Google Drive for rapid updates and file sharing
- Scheduled weekly status check-ins and collaborative work sessions
- Maintained a shared timeline to track progress

Outcome:

Strong team cohesion and role clarity led to smooth collaboration and shared accountability.

Summary:

Despite facing multiple challenges, the team's structured decision-making process enabled timely mitigation strategies. These decisions not only kept the project on schedule but also improved system robustness and enhanced feature quality.

# 17. Conclusions

The development and implementation of the Intelligent Flight Path Planner (IFPP) represents a significant achievement in the intersection of aerospace engineering, artificial intelligence, and software development. Designed to simulate the functionalities of a modern Flight Management System (FMS) within a web-based application, IFPP successfully delivers a lightweight, intelligent, and responsive tool for flight route planning.

Through methodical development and team-based collaboration, the project addressed key challenges in flight operations — namely, route optimization, aircraft capability matching, and real-time weather awareness. The system provides end-users with the ability to select a destination, choose an aircraft, and receive a computed optimal flight path from Dubai International Airport (DXB), while factoring in operational constraints such as aircraft range and live weather data. These features were realized through the integration of OpenWeatherMap and AviationStack APIs, a Flask-based backend for processing, and a Mapbox-powered frontend for visualization.

The final output of the project demonstrates:

- A successful modular system architecture with seamless interaction between frontend, backend, and data APIs
- Logical accuracy in aircraft performance constraints and route validation
- Real-time responsiveness in fetching and displaying environmental data
- User-focused UI/UX design inspired by aviation software principles

Moreover, the project introduced students to the real-world complexities of software testing, version control, error handling, and data-driven engineering workflows. The team demonstrated adaptability, problem-solving, and strong interdisciplinary cooperation — all critical attributes in both aviation and software domains.

Although the IFPP is currently a prototype with simplified logic, it lays the foundation for a future-ready platform with potential applications in pilot training, flight dispatch education, and low-cost airline operations.

## 17.1 Differences between IFPP and a Traditional FMS

While the Intelligent Flight Path Planner (IFPP) draws inspiration from the functionalities of a Flight Management System (FMS), it differs in several critical ways due to its software architecture, intended user base, and scope of application.

**Key Differences:**

1. **Software Platform:**
   A conventional FMS is a certified avionics hardware system embedded within an aircraft's cockpit. In contrast, IFPP is a **web-based application** accessible via browser, requiring no hardware installation or certification, designed for pre-flight planning and educational use.
2. **Operational Scope:**
   Traditional FMS systems integrate in-flight sensor data (e.g., airspeed, position, navigation) and interface directly with autopilot systems. IFPP operates solely **at the pre-flight stage**, focusing on **route suggestion and environmental awareness** without real-time in-flight updates or control over aircraft systems.
3. **Data Integration:**
   An FMS interfaces with proprietary databases updated via ARINC or other certified channels, while IFPP pulls **open-source data from public APIs** like OpenWeatherMap and AviationStack, reflecting a more flexible but less certified data environment.
4. **Complexity and Compliance:**
   Standard FMS must comply with aviation regulatory certifications (FAA, EASA),

while IFPP is a **non-certified, academic-level tool**, prioritizing simplicity and usability over regulatory compliance.

5. **User Base and Purpose:**
   IFPP targets **students, flight enthusiasts, and academic settings** as a learning tool or lightweight operational assistant. A standard FMS is used by **professional pilots in active commercial or general aviation operations** as a critical avionics component.
6. **Cost and Accessibility:**
   Whereas an FMS represents a **multi-million-dollar avionics solution**, IFPP demonstrates a **low-cost, open-source approach** to replicating similar logic for educational, prototype, or low-resource environments.

**Summary:**
IFPP does not intend to replace certified FMS systems but offers a simplified, accessible, and educationally-focused simulation of key flight planning processes — enabling users to experience route generation, weather integration, and aircraft-specific validation without requiring real avionics or in-flight connectivity.

# 18. Future Work and Recommendations

While the current version of the Intelligent Flight Path Planner (IFPP) successfully achieves its core objectives, there remains significant scope for functional and technical enhancement. Future iterations of the system can focus on expanding its utility, improving its accuracy, and incorporating advanced decision-support capabilities consistent with real-world aviation systems.

## 18.1 Future Development Opportunities

1. Fuel Consumption and Payload Estimation Module
   - Future versions should integrate real-time fuel consumption calculators based on aircraft-specific drag, thrust, and altitude performance profiles. Payload and center-of-gravity impact could also be modeled for more realistic dispatch simulation.
2. AI-Powered Autopilot and Auto-Routing
   - Integration of a basic AI decision engine to simulate autopilot routing that dynamically reroutes based on real-time weather, NOTAMs (Notices to Airmen), and airspace closures would significantly enhance realism and usability.
3. Synthetic Vision and Terrain Awareness

- Implementation of 3D terrain mapping and obstacle visualization (akin to Garmin SVT) would improve situational awareness, especially for general aviation users or in training environments.

4. Support for IFR Route Planning and ATC Simulation
    - Future versions could introduce en-route airways, SIDs, STARs, and simulate ATC advisories, enabling instrument flight rule (IFR)-based flight plan generation.
5. Offline Capability and Mobile App Porting
    - Developing a mobile or tablet version (e.g., using Swift for iPadOS) would broaden user accessibility. Incorporating offline data caching (airports, METAR history) could also be useful in disconnected environments.

## 18.2 Recommendations for Future Teams and Stakeholders

| Recommendation | Justification |
|---|---|
| Adopt Agile Methodology with Sprint Reviews | Increases adaptability and ensures progress through iterative testing and user feedback |
| Collaborate with Aviation Professionals | Insights from pilots or flight planners can refine performance models and interface logic |
| Use Cloud Infrastructure for Backend Hosting | Transitioning to a cloud service (e.g., AWS, Heroku) would support real-time deployment, API scaling, and version control |
| Automate Testing and Error Logging | Unit tests, logging tools, and backend health checks will improve system reliability |
| Expand to Multi-Airport Dispatch Mode | Allowing multiple origin airports or connecting flights adds realism for large-scale simulations |

Closing Note:

The IFPP project has demonstrated the feasibility and practicality of an intelligent, AI-assisted flight planning tool within an educational context. Future work should focus on transforming this prototype into a professional-grade platform capable of contributing to the future of digital aviation and pilot decision support systems.

# 19. Acknowledgements

The team would like to express sincere gratitude to all individuals and institutions whose support made the successful completion of the Intelligent Flight Path Planner (IFPP) project possible.

First and foremost, we extend our heartfelt thanks to Dr. Walid, our project supervisor, for his consistent guidance, valuable technical insights, and constructive feedback throughout the semester. His mentorship played a crucial role in shaping the project's direction and refining its deliverables.

We would also like to thank the faculty and staff at Emirates Aviation University, particularly those from the School of Engineering, for providing the academic framework, resources, and learning environment that helped the development of this interdisciplinary project.

Special thanks are due to:

- Online API providers such as OpenWeatherMap and AviationStack, whose accessible data services formed the backbone of our real-time system.
- Figma, GitHub, Mapbox, and the Flask open-source communities for making powerful and flexible development tools freely available.
- Our classmates and peers who offered user feedback during testing phases, helping us improve the app's functionality and interface clarity.

Finally, we would like to acknowledge the commitment and hard work of all team members — Sadish, Hair, and Ali Tariq — whose collaborative spirit, mutual respect, and technical contributions brought this project to life.

# 20. References

### Book-Related References

- Nielsen, J. (1995). Usability Engineering. San Diego: Academic Press.
- Endsley, M. R. (2000). Designing for Situation Awareness: An Approach to User-Centered Design. CRC Press.


## Journal References


- Kalaboukas, K., Katsoulakos, P., & Tsakalidis, A. (2019). 'Improving Flight Planning using Adaptive AI Algorithms', Journal of Aerospace Operations, 7(1), pp. 34–49.
- Zhang, H., Liu, Y., & Gao, J. (2020). 'Trajectory Prediction in Air Traffic Management using Deep Neural Networks', IEEE Transactions on Intelligent Transportation Systems, 21(5), pp. 2042–2051.
- Prats, X., Pastor, E., & Royo, P. (2014). 'Fuel-efficient en-route trajectory planning in presence of wind', Transportation Research Part C: Emerging Technologies, 40, pp. 1–13.
- Liu, M., Zeng, L., & Wu, T. (2021). 'Weather-based Risk Assessment for Airline Route Selection', Aerospace Science and Technology, 115, 106827.
- Ferreira, D., Carvalhal, A., & Antunes, D. (2020). 'Web Services in Aerospace: RESTful Approaches for Live Aviation Data Integration', International Journal of Aviation Technology and Engineering, 9(3), pp. 77–90.
- Rego, J., Li, Z., & Stewart, D. (2018). 'An Open-Source Web-Based Flight Planner for General Aviation', Journal of Aerospace Information Systems, 15(4), pp. 227–238.


## Internet Website References


- OpenWeatherMap. (n.d.). Weather API for developers. Available at: https://openweathermap.org/api
- AviationStack. (n.d.). Real-time Flight Status and Aviation Data API. Available at: https://aviationstack.com
- Mapbox. (n.d.). An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps.
- GitHub. (n.d.). The world's leading open-source version control platform. Available at: https://github.com
- Figma. (n.d.). Collaborative Interface Design Tool. Available at: https://www.figma.com
- Flask. (n.d.). A micro web framework for Python. Available at: https://flask.palletsprojects.com